

```

#include <iostream>
using namespace std;

const int MAX = 100;
const int INF = 999999999;

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = (i == j) ? 0 : INF;
    cout << "Enter edges (Source Destination Weight):\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n &&
            u != v && adj[u - 1][v - 1] == INF) {
            adj[u - 1][v - 1] = w;
            adj[v - 1][u - 1] = w; // undirected: both directions
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

// Prim's Algorithm - builds MST starting from a given node
void primMST(int adj[MAX][MAX], int n, int startNode) {
    int key[MAX]; // min edge weight to connect node into MST
    int parent[MAX]; // stores MST tree structure
    bool inMST[MAX]; // true if node already included
    for (int i = 0; i < n; i++) {
        key[i] = INF;
        inMST[i] = false;
        parent[i] = -1;
    }
    key[startNode] = 0;
    for (int count = 0; count < n; count++) {
        // pick min-key node not yet in MST
        int u = -1, minVal = INF;
        for (int i = 0; i < n; i++) {
            if (!inMST[i] && key[i] < minVal) {
                minVal = key[i];
                u = i;
            }
        }
        if (u == -1) break; // remaining nodes unreachable
        inMST[u] = true;
        // update keys of adjacent nodes
        for (int v = 0; v < n; v++) {
            if (adj[u][v] != INF && !inMST[v] && adj[u][v] < key[v]) {
                key[v] = adj[u][v];
                parent[v] = u;
            }
        }
    }
}

```

```

    cout << "\nEdge \tWeight\n";
    int totalWeight = 0;
    for (int i = 0; i < n; i++) {
        if (parent[i] != -1) {
            cout << parent[i]+1 << " - " << i+1 << "\t" << key[i] << "\n";
            totalWeight += key[i];
        }
    }
    for (int i = 0; i < n; i++)
        if (!inMST[i])
            cout << "Node " << i+1 << " is unreachable.\n";
    cout << "\nTotal MST Weight = " << totalWeight << endl;
}

int main() {
    int n, e;
    int adj[MAX][MAX];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(adj, n, e);
    int start;
    cout << "\nEnter starting node: ";
    cin >> start;
    primMST(adj, n, start - 1);
    return 0;
}

```

WHEN TO USE PRIM'S (Scenario Triggers)

- ▶ Question asks for Minimum Spanning Tree (MST) — connect all nodes at minimum total cost
- ▶ Graph is DENSE (many edges, close to complete graph) — class notes: "Prim's — node [Dense Graph]"
- ▶ "Connect all cities/offices/computers with minimum total cable/road cost" style scenarios
- ▶ You already know the graph is CONNECTED (Prim's naturally grows one tree from a start node)
- ▶ Question gives weighted, undirected edges and asks to minimize total connection cost

Prim's vs Kruskal's — Which One?

- ▶ Graph is DENSE → Prim's is more efficient (grows tree node-by-node)
- ▶ Graph is SPARSE or edges given as a simple list → Kruskal's (sorts edges, uses Union-Find)
- ▶ Need to pick edges in strict sorted order? → Kruskal's
- ▶ Both ALWAYS give the same total MST weight — just build it differently
- ▶ Prim's grows ONE connected tree outward. Kruskal's can grow MULTIPLE separate pieces that merge later

MANUAL TRACE METHOD (Selected-Node Table, from Class Notes)

Use this exact process — it matches the "Selected node" table format from class:

- ▶ 1. Make a table: rows = selected nodes so far, columns = ALL nodes in the graph.
- ▶ 2. Start: mark the start node's row with 0 for itself, INF (α) for all others.
- ▶ 3. Pick the SMALLEST unvisited value in the current row → that becomes the next selected node.
- ▶ 4. New row: copy previous row, but UPDATE any column where a smaller edge weight exists from the newly selected node.
- ▶ 5. Circle the value you just picked (this becomes part of the MST).
- ▶ 6. Repeat until every node has been selected (circled) once.
- ▶ 7. Sum all the circled (selected) values = Total MST weight.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Undirected requirement — Prim's REQUIRES $\text{adj}[u][v] = \text{adj}[v][u]$ (both directions set). Forgetting this breaks the algorithm silently.

Disconnected graph — if some node's $\text{key}[]$ stays INF forever, it's unreachable. MST is only valid for connected graphs; say so explicitly if asked.

MST is NOT unique if there are equal-weight edges — multiple valid MSTs can exist, but total weight is always the same.

Don't confuse 'key[]' (cheapest edge INTO the MST) with 'shortest path from source' (that's Dijkstra) — Prim's minimizes EDGE weight, not cumulative distance.

n nodes → MST always has exactly $n-1$ edges. If your edge count doesn't match, something's wrong.

EXAMPLE 1 — Small Graph (Trace by Hand)

Graph (undirected, weighted): A-B=2, A-C=3, B-C=1, B-D=4, C-D=5

Start node: A

Step-by-step trace:

- ▶ $\text{key}[A]=0$, $\text{key}[B]=\text{INF}$, $\text{key}[C]=\text{INF}$, $\text{key}[D]=\text{INF}$
- ▶ Select A ($\text{key}=0$) → update: $\text{key}[B]=2$ (A-B), $\text{key}[C]=3$ (A-C)
- ▶ Select B ($\text{key}=2$, smallest) → update: $\text{key}[C]=\min(3,1)=1$ (B-C better), $\text{key}[D]=4$ (B-D)
- ▶ Select C ($\text{key}=1$, smallest) → update: $\text{key}[D]=\min(4,5)=4$ (no change, B-D still better)
- ▶ Select D ($\text{key}=4$) → done, all nodes selected

MST Edges: A-B(2), B-C(1), B-D(4) → Total Weight = 7

EXAMPLE 2 — Disconnected Graph

Graph: A-B=5, C-D=3 (two separate components)

Start from A → only A, B get included (key stays INF for C, D forever).

Exam expects you to say: "No MST exists for the full graph since it's disconnected. Node C and D are unreachable from A." Prim's cannot fix this by restarting (unlike BFS/DFS) — it only builds ONE tree from ONE component.

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "An ISP wants to lay fiber cable connecting 6 towns with minimum total cable length. Cable cost between any two towns is given."

How to apply Prim's: towns = nodes, possible cable routes = weighted edges (cost = length/price). Since this is likely a DENSE graph (cost given between most/all town pairs), Prim's is the efficient choice over Kruskal's. Start from any town, grow the MST — the final total weight is the minimum total cable cost.

EXAMPLE 4 — Tie-Breaking Twist (Medium/Hard style)

Scenario: two different edges have the EXACT same weight during selection (e.g., $\text{key}[C]=4$ and $\text{key}[D]=4$ at the same step).

Correct handling: either choice is valid — both lead to a valid MST with the same TOTAL weight, though the specific edges included may differ. If the exam asks "is the MST unique?", the answer is NO when tie-breaking edges exist — always check for equal-weight edges before claiming uniqueness.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Prim's answer, check:

Did I treat the graph as undirected (set both $\text{adj}[u][v]$ and $\text{adj}[v][u]$)?

Did I pick the SMALLEST key value at each step, not the smallest node index?

Does my final edge count equal $n-1$ (n = number of nodes)?

If the graph is disconnected, did I state that no full MST exists?

Did I check for tied edge weights before claiming the MST is unique?